

구조 기반 산출물 생성 활동 — 실습

"jsondb를 만들어줘"라고 요청했더니 일어난 일

Ep. 05 / 14



배움의 활동 (Ep4):

Education → Knowledge Capture → Reflection



Learning →



Generation



만듦의 활동 (Ep5):



ADR → Spec → Generation → Verification

여러분, AI에게 “이것 좀 만들어줘”라고 요청한 후, 그대로 쓰신 적 있나요?

AI의 제안이 항상 내 요구와 맞는 것은 아니다.

- ☑️ 🏗️ **[Architecture Design]**: AI와 함께 구조 기반 산출물 생성의 핵심 개념 이해.
- ☑️ 🔍 **[AI Analysis]**: AI 제안이 부적절한 3가지 이유 (일반적이다 / 쉽고 간단하다 / 품질 미반영)를 분석.
- ☑️ 📄 **[Living Documentation]**: ADR과 Spec이 함께 진화하는 살아있는 문서 개념 정립.
- ☑️ 😬 **[Hidden Decisions]**: Undocumented ASR(암묵적 설계 결정)의 개념과 검증의 중요성 습득.

[ADR] → [Spec] → [Gen] → [Verification]

Core Path of This Practice

“jsondb를 만들어줘” — 하지만 AI는 설계부터 권했다

50%

일반 AI (Standard AI)



jsondb? 알겠습니다!

와르르 효과

즉시 수백 줄 코드 생성
(Immediate Code Generation)

검토 없이 쏟아내는 코드



Cocrates

50%

"이 jsondb는 어떤 용도인가요?
아키텍처를 먼저 설계하고
검토받은 후 생성하겠습니다."



설계 없는 생성을 거부한다
(Refuses Unstructured Generation)

Ep2 원칙 : “검토되지 않은 산출물은 생성할 가치가 없다.”

ADR ① — 저장 모델 논쟁

 **AI의 제안:** MongoDB처럼 Collection-Document 구조가 일반적이고 쉽습니다

 **핵심 문제: 일반적이다 (Generic)**

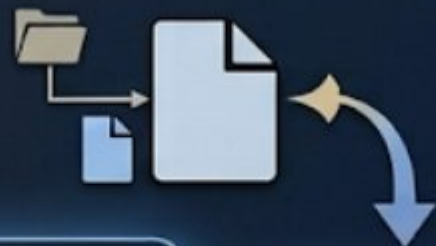


Collection
Document
Document



 **사용자의 브레이크:** 내가 지정한 경로가 그대로 파일명이 되는 Path-Addressable 방식이 필요해

예: `Set("episode/e1")`
→ 실제 `episode/e1.json` 파일



 **결과:** 사용자의 요구대로 경로 매핑 구조로 결정

 **교훈:** 일반적이다 ≠ 내 요구에 맞다. AI가 가장 흔히 접한 패턴이 최적은 아니다.

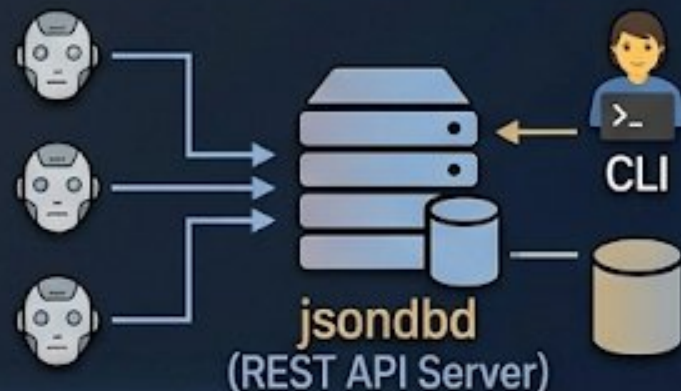
ADR ② — 아키텍처 논쟁

 **AI의 제안:** “단일 프로세스 Library-Only가 쉽고 간단합니다”

 **핵심 문제:** 쉽고 간단하다 (Simple)



 **사용자의 브레이크:** “여러 AI 에이전트가 동시에 접근할 거야. Client-Server 구조로 가야 해.”



 **결과:** REST API 서버(jsondbd) + CLI 도구 포함 아키텍처로 전면 수정

 **교훈:** 쉽고 간단하다 ≠ 올바른 설계다. AI가 만들기 쉬운 방식이 실제 환경의 요구를 만족하지 않을 수 있다.

ADR ③ — 동시성 모델 논쟁

 **AI의 제안:** “DB 전체 락(DB-level Lock)이면 코드 3줄이면 끝납니다”

 **핵심 문제: 품질 (Quality-blind)**



DB-level Lock

```
db.Lock(); // Lock entire DB
do_something();
db.Unlock();
```

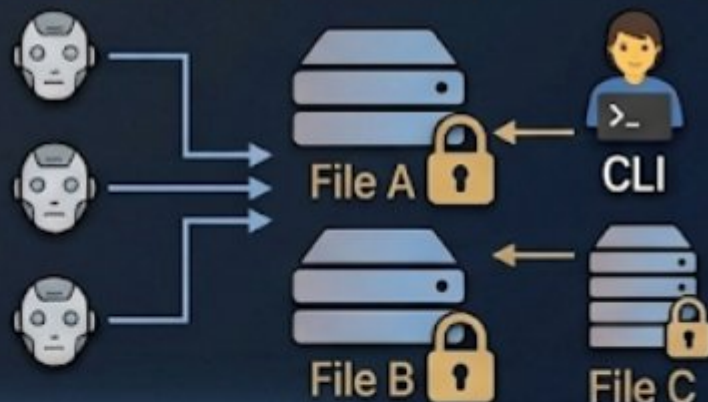
DB Lock

비교
(Comparison)



File Mutex

 **사용자의 브레이크:** “파일 하나 읽을 때 전체 DB를 멈추면 병목이잖아! 파일별 락(Per-File Mutex)으로 해줘.”



 **결과: sync.Map을 활용한 고성능 동시성 제어**

 **교훈:** AI는 품질을 충분히 반영하지 못한다. 성능, 확장성 같은 품질 속성은 사용자가 지적해야 반영된다.

ADR 교훈 — “AI의 제안은 항상 내 요구와 맞는 것은 아니다”

ADR ① 저장 모델 논쟁



- 일반적이었지만, 내 요구사항과 맞지 않았다

ADR ② 아키텍처 논쟁

- 쉽고 간단했지만, 미래 확장성을 담지 못했다



ADR ③ 동시성 모델 논쟁

- 품질(성능)을 반영하지 못했다



AI 제안의 부적절함은 다양한 형태로 나타난다:



할루시네이션

VS



일반적 패턴



쉽고 간단



품질 무시

- 올바른 AI 사용을 위한 아키텍처 설계 안내
- 사용자의 구조 설계 역량 향상
- U→A→E→A 검토 문화 정착



Cocrates의 역할:

(U→A→E→A 검토 문화)

→ 사용자의 검토와 승인이 필수적이다

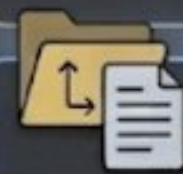


ADR도 살아있는 문서다

- 요구사항 변화에 따라 결정 변경 가능

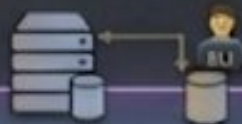
Spec — 결정을 하나의 헌법으로

ADR ① 저장 모델 논쟁



ADR ② 아키텍처 논쟁

ADR ③ 동시성 모델 논쟁



spec/jsondb.md



spec/jsondb.md

-  10개 REST 엔드포인트 기능
-  API 시그니처와 CLI 명령어 스펙
-  테스트 요구사항
-  Self-Containment (자기 완결성)

 Spec =
설계의 헌법

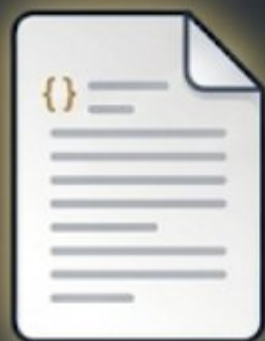
유일한 진실 공급원
(Single Source of Truth)

 초기의 모호한 요청은 맞힌다 (맞힌요청)

 포인트: Spec 없이 생성은 있을 수 없다.
Spec이 곧 설계의 완성이다.

⚡ Generation — 오직 Spec만 보고 달린다

Spec



spec/jsondb.md

Generation



모드: ⚡ spec-driven-generation 활성화

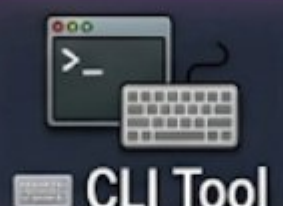
✓ 컴파일 오류 0개
(go build, go vet 패스)



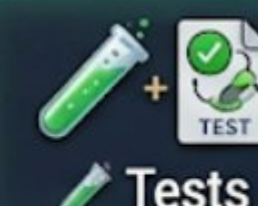
Core Lib



REST API



CLI Tool



Tests

❌ 초기 프롬프트("jsondb 만들어줘")

✓ 합의된 Spec만 유일한 법전

핵심 메시지: "Spec을 잘 설계하면 AI가 코드를 잘 만든다.
Spec의 품질이 결과물의 품질을 결정한다."

Verification — 설계 검증 + 동시에 설계 재검토

역할 1: "사양대로 구현되었는가?"



결과: ✓ 71개 PASS, ✗ 1개 FAIL (Deviation)

발견된 Deviation:

Spec: `PUT /schema/blog/episode`
코드: `PUT /schema//blog/episode`
(이중 슬래시)

복잡도가 높을수록 AI가 사양을 놓칠 가능성은 높아진다
→ 작은 단위로 점진적/반복적 개발 필요



큰 Spec 한 번 구현 → 검증 부담 폭증.
작은 Spec씩 구현 → 검증+피드백 선순환

역할 2: "설계 자체가 올바른가?"



- 검증하면서 동시에 설계의 올바름을 재검토
- "사양대로 안 되면 AI 잘못" ✗
→ "설계 자체를 변경해야 하는 상황" 일 수 있음

포인트: "Verification은 단순한 체크리스트가 아니다. Spec 확인 + 설계 재검토의 기회다."

🤖 Undocumented ASR — AI의 침묵의 기본값

역할 1: "사양대로 구현되었는가?"



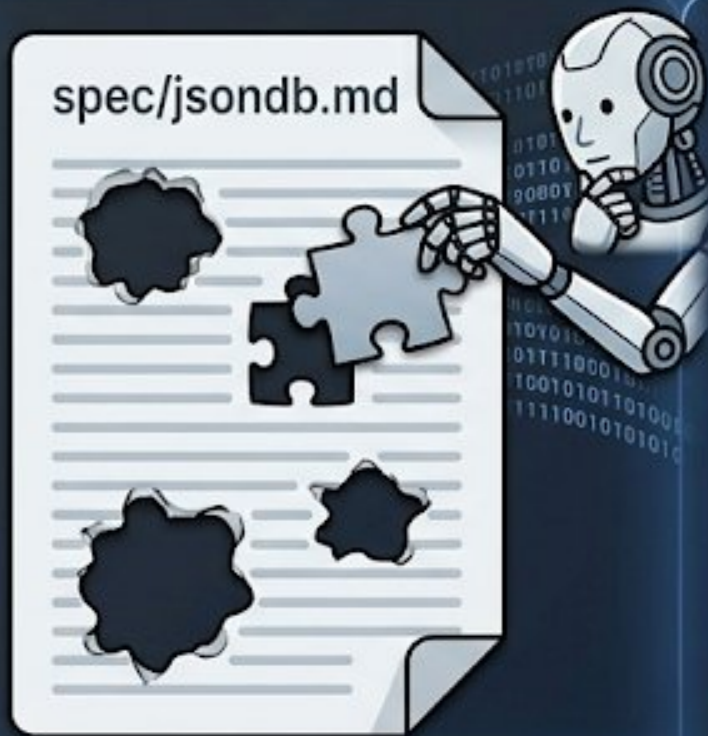
72



결과: ✅ 71개 PASS, ❌ 1개 FAIL (Deviation)

검증 중 6건 발견:

- 🔗 URL 인코딩 누락
- 🔄 데이터 수정 시 스키마 재검증 생략



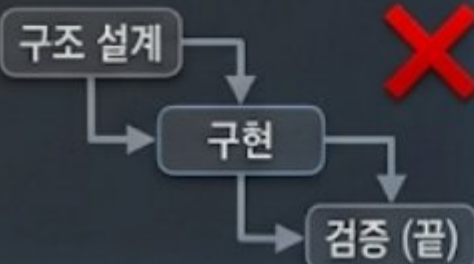
검증 중 6건 발견:

- 🔗 URL 인코딩 누락
- 🔄 데이터 수정 시 스키마 재검증 생략
- ❌ 無 無결성 검증 누락
- 🛡️ 권한 관리 부족
- 📄 데이터 타입 불일치
- ❌ 에러 핸들링 미흡

💡 교훈: 검증을 통해 빈틈을 발견하고 Spec에 추가해야 한다.

구조는 고정되지 않는다 — Living Cycle

오해: Waterfall



현실: Living Cycle

검증 피드백 회귀



요구사항 변경

검증 발견



핵심 개념:

ADR도 살아있는 문서다

Spec도 살아있는 문서다

포인트: “구조 기반 개발은 Waterfall이 아니다. 구조가 개발을 이끌고, 개발 경험이 구조를 개선한다.”

핵심 요약 + 학습 확인

🔍 AI의 제안을 검토하라.

할루시네이션만 문제가 아니다.
 AI 제안은 다양하게
 부적절할 수 있다 —
일반적이어서, 쉽고 간단해서,
 또는 품질을 반영하지 못해서.

📄 ADR과 Spec은 함께 살아있는 문서다.

검증/요구사항 변경 시 
 구조 설계(ADR)로 돌아가
 결정을 재검토한다.
 구조는 한 번 정해지면
 변하지 않는 것이 아니다.

✅ 검증이 곧 품질이다.



72개 검증으로도 모든 걸
 잡을 수 없지만,
 검증 없이는 그조차
 발견할 수 없다.

🦉 Cocrates는 여러분의 구조 설계 역량을 키우는 도구다. →

단순 코드 생성기가 아니다.
 올바르게 AI를 사용하고
 스스로 설계 능력을
 키우도록 안내한다.

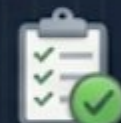
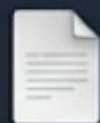
? 학습 확인 (자기 질문)

- ? Artifact Generation 파이프라인의 4단계를 말할 수 있는가?
- ? AI의 제안이 왜 내 요구와 맞지 않는지, 세 가지 이유를 실제 사례와 함께 설명할 수 있는가?
- ? Undocumented ASR이 무엇이고 왜 발생하는지 설명할 수 있나요?
- ? 구조 기반 개발이 왜 Waterfall이 아닌지, ADR과 Spec이 어떻게 함께 진화하는지 말할 수 있나요?



오늘 배운 것: 구조 기반 생성 파이프라인

ADR → Spec → Generation → Verification



구조 기반 파이프라인

Ep. 6

다음 편 (Ep. 6): 구조 기반 워크플로우
생성 실습 — “스킬을 만드는 스킬”



남이 만든 스킬을 넘어, 우리가 직접 새로운
새로운 스킬을 Cocrates와 함께 만든다



설명문/보고서 자동화 스킬을 직접 생성



“지금까지 AI가 짜준 첫 번째 코드 제안을,
나는 아무 의심 없이 복사·붙여넣기 하진 않았는가?”



오늘 만들 것 (Today's Project): jsondb — 로컬 JSON 저장소

